

Proyecto Final: Compilador de Rust a x86-64

CS3402 - Compiladores | UTEC

Semestre: 2026-1

Informe técnico actualizado con aplicación integrada, benchmarks contra rustc/GCC, métricas de optimización y capturas del pipeline visual.

1. Resumen

Se implementó un compilador para un subconjunto educativo de Rust que genera ensamblador x86-64 (AT&T, System V ABI). El pipeline incluye análisis léxico, sintáctico, semántico, generación de código y optimización (DAG + Peephole + constant folding). Se añadieron inferencia de tipos, strings, arreglos 2D y una aplicación web con seis pestañas que visualiza tokens, AST, assembly, simulación y estadísticas.

2. Arquitectura

Flujo: Código fuente → Scanner → Parser → AST → TypeChecker → GenCodeVisitor → Optimizer → archivo .s. La API expone --emit-json para la app Flask (puerto 5002).

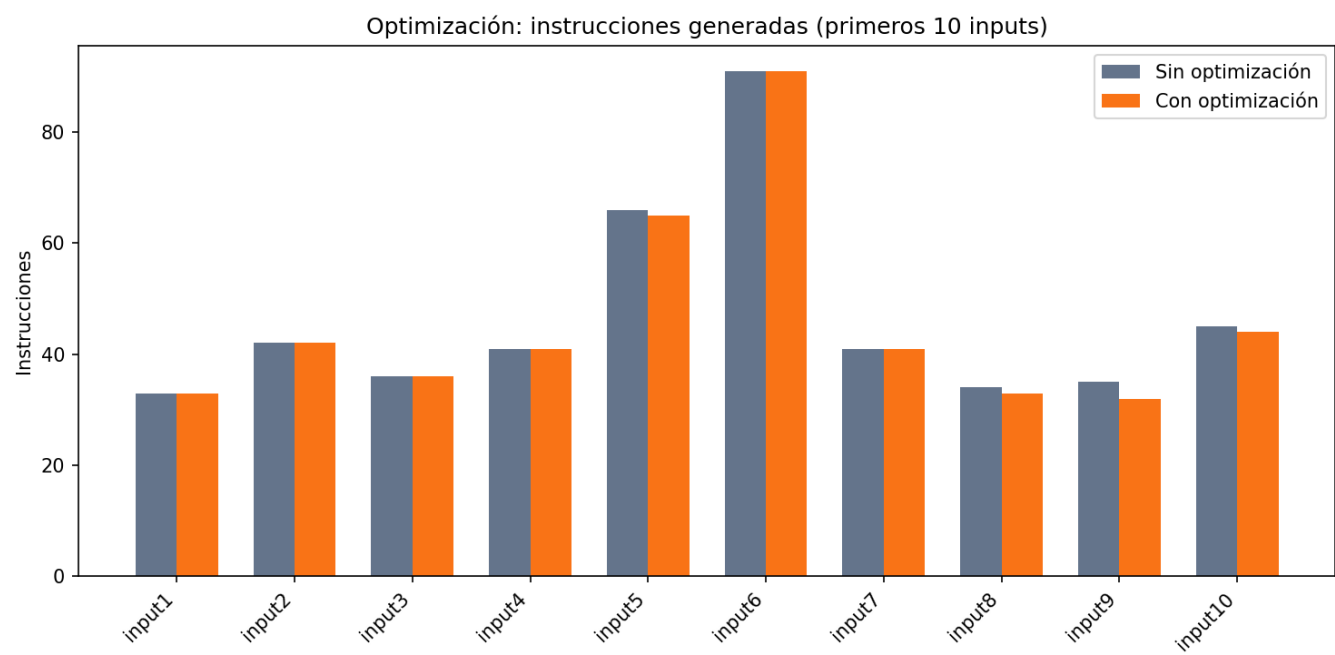
3. Características avanzadas implementadas

Característica	Estado	Ejemplo
Inferencia de tipos	Sí	let x = 5;
Strings	Sí	let s: String = "Ho...
Arreglos 2D	Sí	let mut m: i32[2][3];
Punteros / genéricos	No	Trabajo futuro

4. Optimización

Se midió la reducción de instrucciones assembly con y sin optimizador en 21 inputs. Total de instrucciones ahorradas: 11. Las técnicas aplicadas son: eliminación de subexpresiones comunes (DAG), peephole (strength reduction, mov redundante) y constant folding.

Figura 1. Instrucciones con/sin optimización

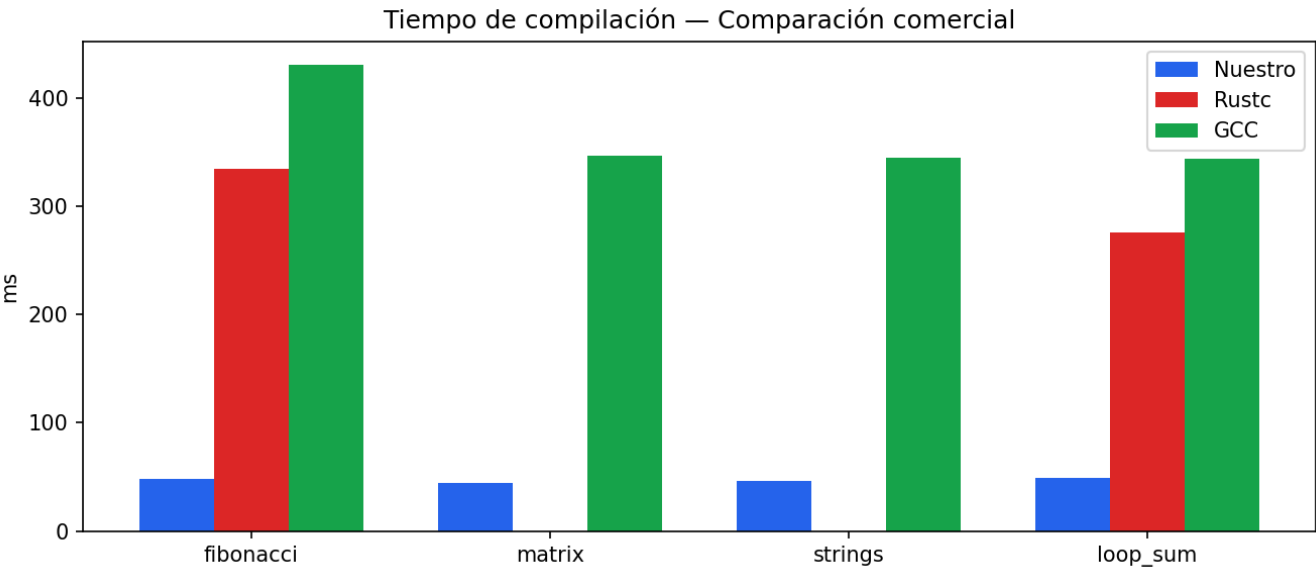


5. Comparación comercial

Se comparó el tiempo de compilación (promedio de 5 ejecuciones) contra rustc y GCC. Para GCC se usaron programas equivalentes en C con la misma lógica. Nuestro compilador es más rápido en compilación porque implementa un subconjunto reducido sin backend LLVM ni borrow checker.

Programa	Nuestro	Rustc	GCC	Ratio R	Ratio G
fibonacci.rs	48.6	334.3	430.6	6.88	8.86
matrix.rs	44.9	N/A	346.2	N/A	7.72
strings.rs	46.8	N/A	344.3	N/A	7.36
loop_sum.rs	49.4	275.5	344.0	5.58	6.96

Figura 2. Tiempos de compilación (ms)



5.1 Por qué matrix y strings no compilan en rustc

Los benchmarks matrix.rs y strings.rs usan la sintaxis de nuestro dialecto educativo, no Rust estándar. Rustc rechaza matrix.rs porque escribimos let mut m: i32[3][3]; cuando Rust exige let mut m: [[i32; 3]; 3];. En strings.rs, asignar un literal directamente a String falla (rustc espera &str o .to_string()). Esto no indica un fallo de nuestro compilador: medimos rustc solo donde la sintaxis coincide (fibonacci, loop_sum). GCC compila los equivalentes en C para los cuatro casos.

6. Aplicación integrada (6 pestañas)

La app web en <http://127.0.0.1:5002> permite compilar desde el navegador y recorrer Código → Tokens → AST → Assembly → Simulación → Salida & Stats.

Figura 3. Pestaña Código

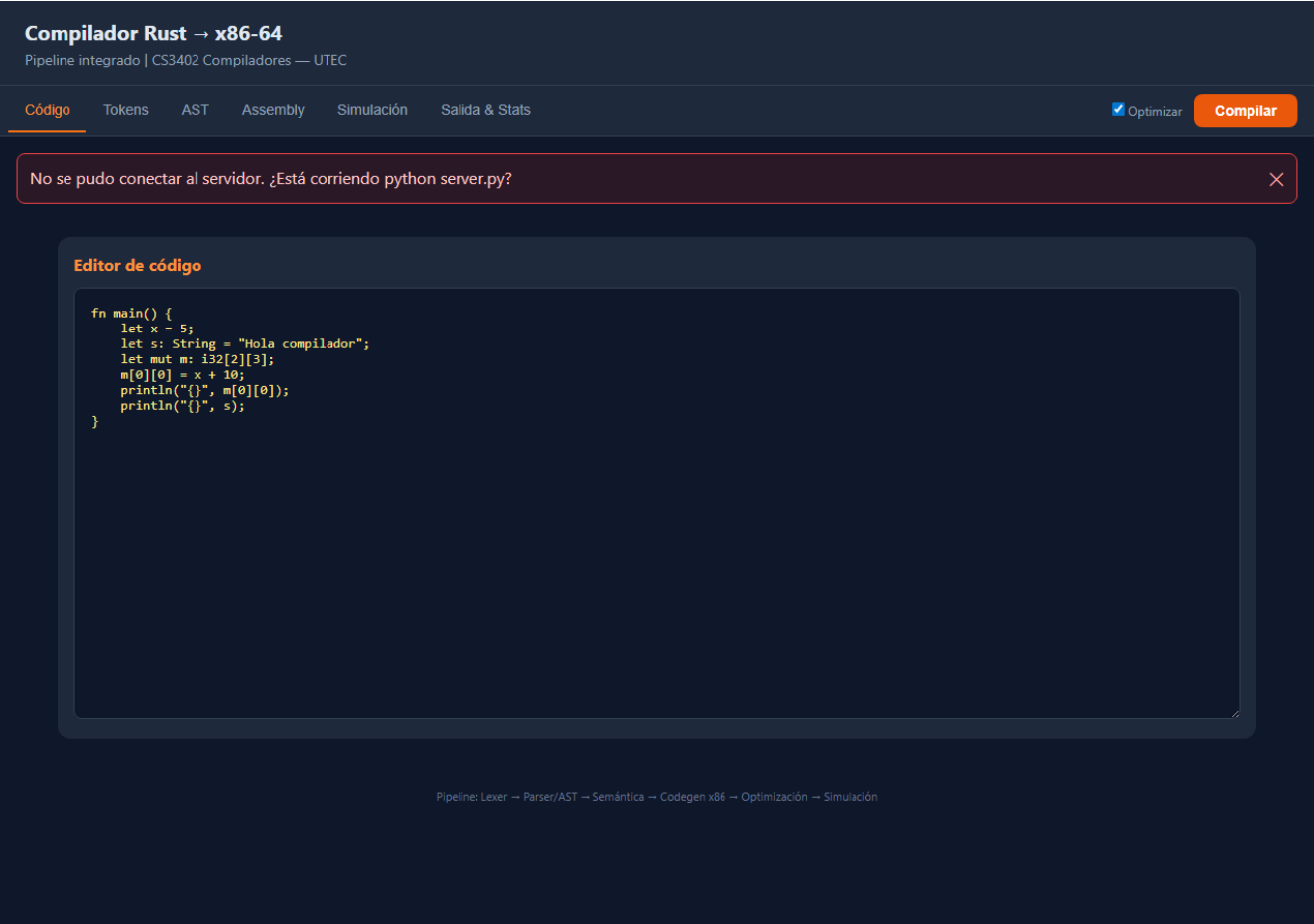


Figura 4. Pestaña Tokens

No se pudo conectar al servidor. ¿Está corriendo python server.py? ✕

Análisis Léxico — Tokens

#	Tipo	Texto
1	FN	fn
2	IDENTIFIER	main
3	LPAREN	(
4	RPAREN	)
5	LBRACE	{
6	LET	let
7	IDENTIFIER	x
8	ASSIGN	=
9	NUMBER	5
10	SEMICOL	;
11	LET	let
12	IDENTIFIER	s
13	COLON	:
14	STRING	String
15	ASSIGN	=
16	STRING_LITERAL	"Hola compilador"
17	SEMICOL	;
18	LET	let
19	MUT	mut
20	IDENTIFIER	m
21	COLON	:
22	i32	i32
23	LBRACKET	[
24	NUMBER	2
25	RBRACKET	]
26	LBRACKET	[
27	NUMBER	3
28	RBRACKET	]
29	SEMICOL	;
30	IDENTIFIER	m
31	LBRACKET	[
32	NUMBER	0
33	RBRACKET	]
34	LBRACKET	[
35	NUMBER	0
36	RBRACKET	]
37	ASSIGN	=
38	IDENTIFIER	x
39	PLUS	+
40	NUMBER	10
41	SEMICOL	;
42	PRINTLN	println
43	LPAREN	(
44	STRING_LITERAL	"[]"
45	COMMA	,
46	IDENTIFIER	m
47	LBRACKET	[
48	NUMBER	0
49	RBRACKET	]
50	LBRACKET	[
51	NUMBER	0
52	RBRACKET	]
53	RPAREN	)
54	SEMICOL	;
55	PRINTLN	println
56	LPAREN	(
57	STRING_LITERAL	"[]"
58	COMMA	,
59	IDENTIFIER	s
60	RPAREN	)
61	SEMICOL	;
62	RBRACE	}

Figura 5. Pestaña AST

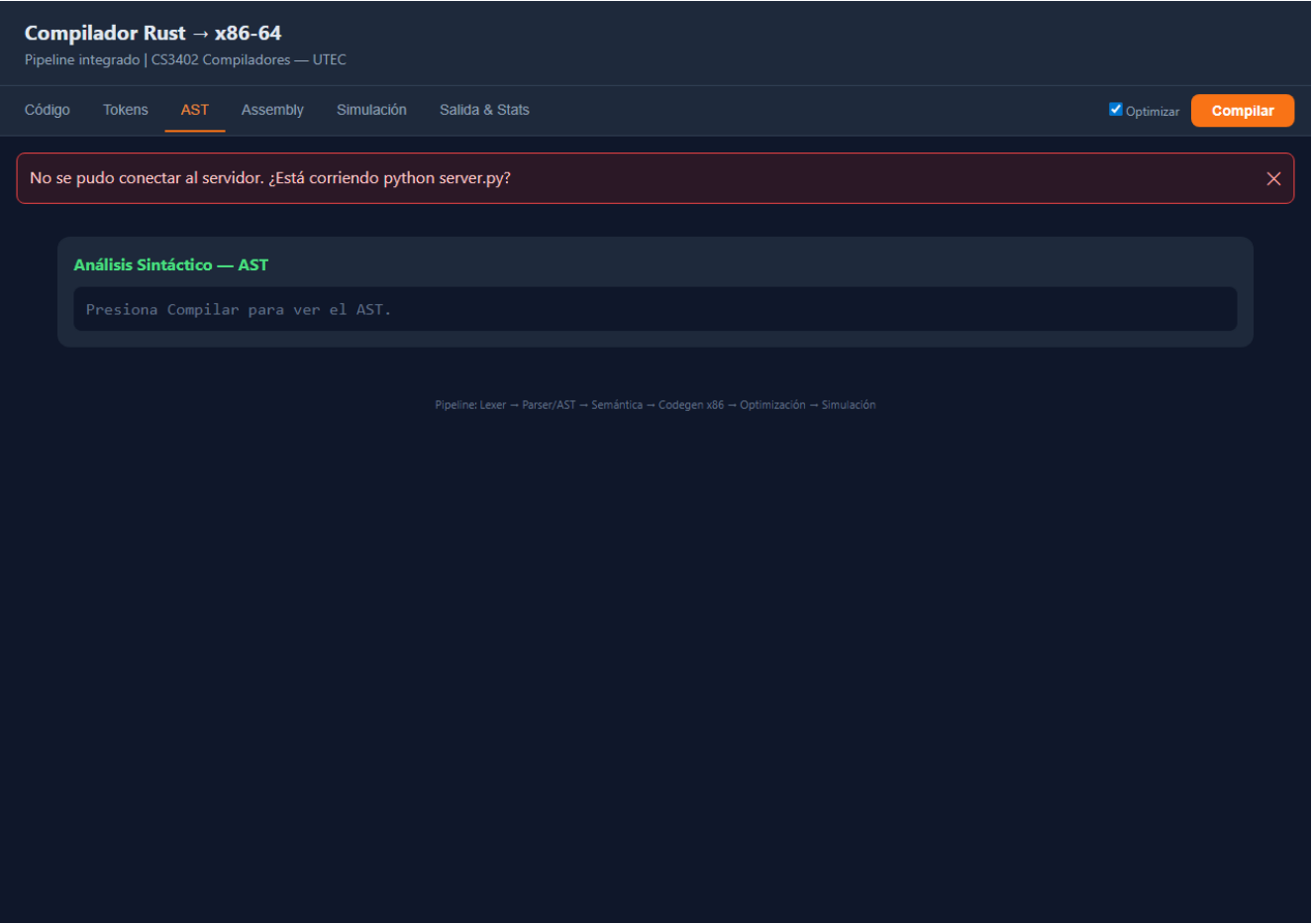


Figura 6. Pestaña Assembly

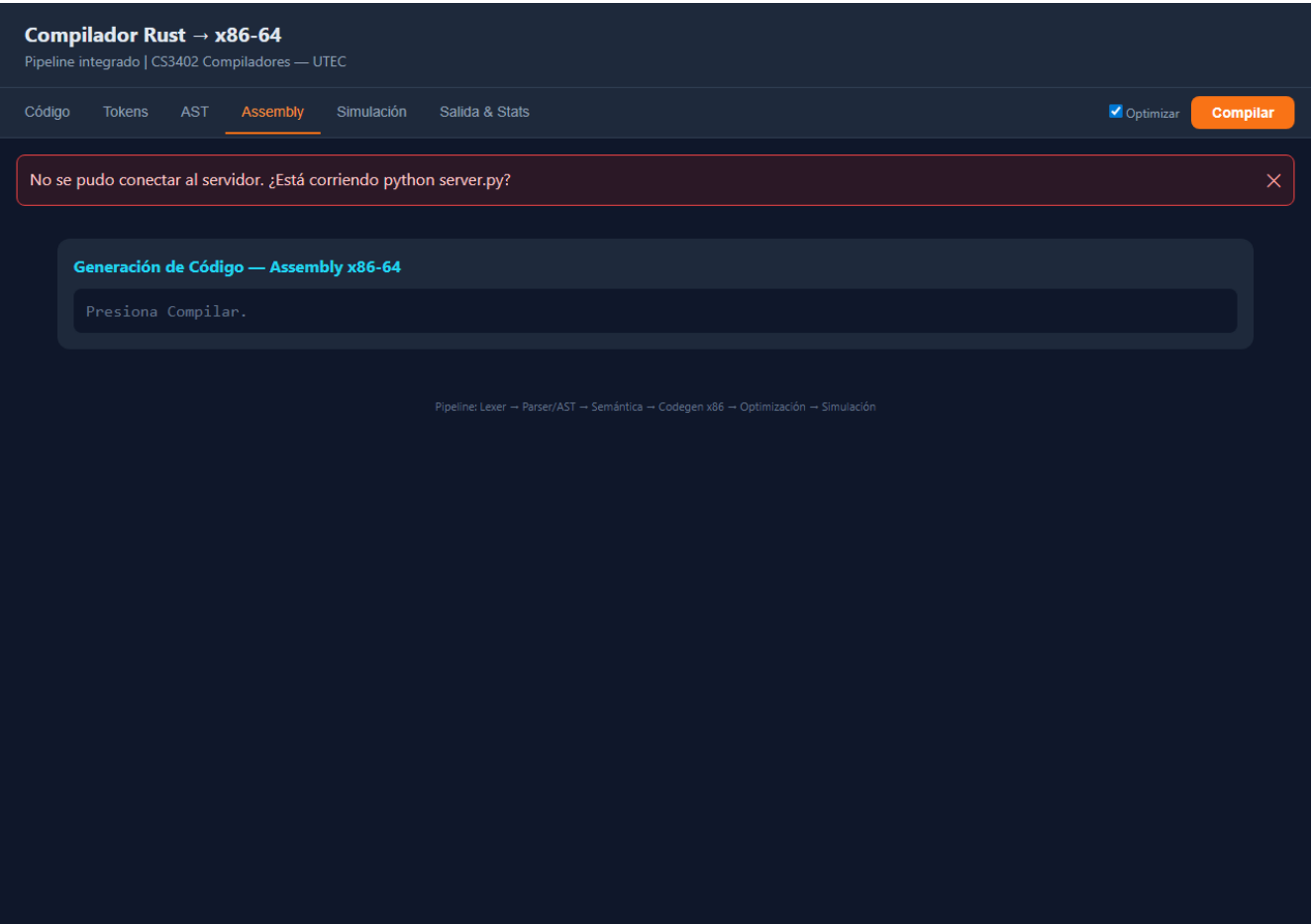


Figura 7. Pestaña Simulación

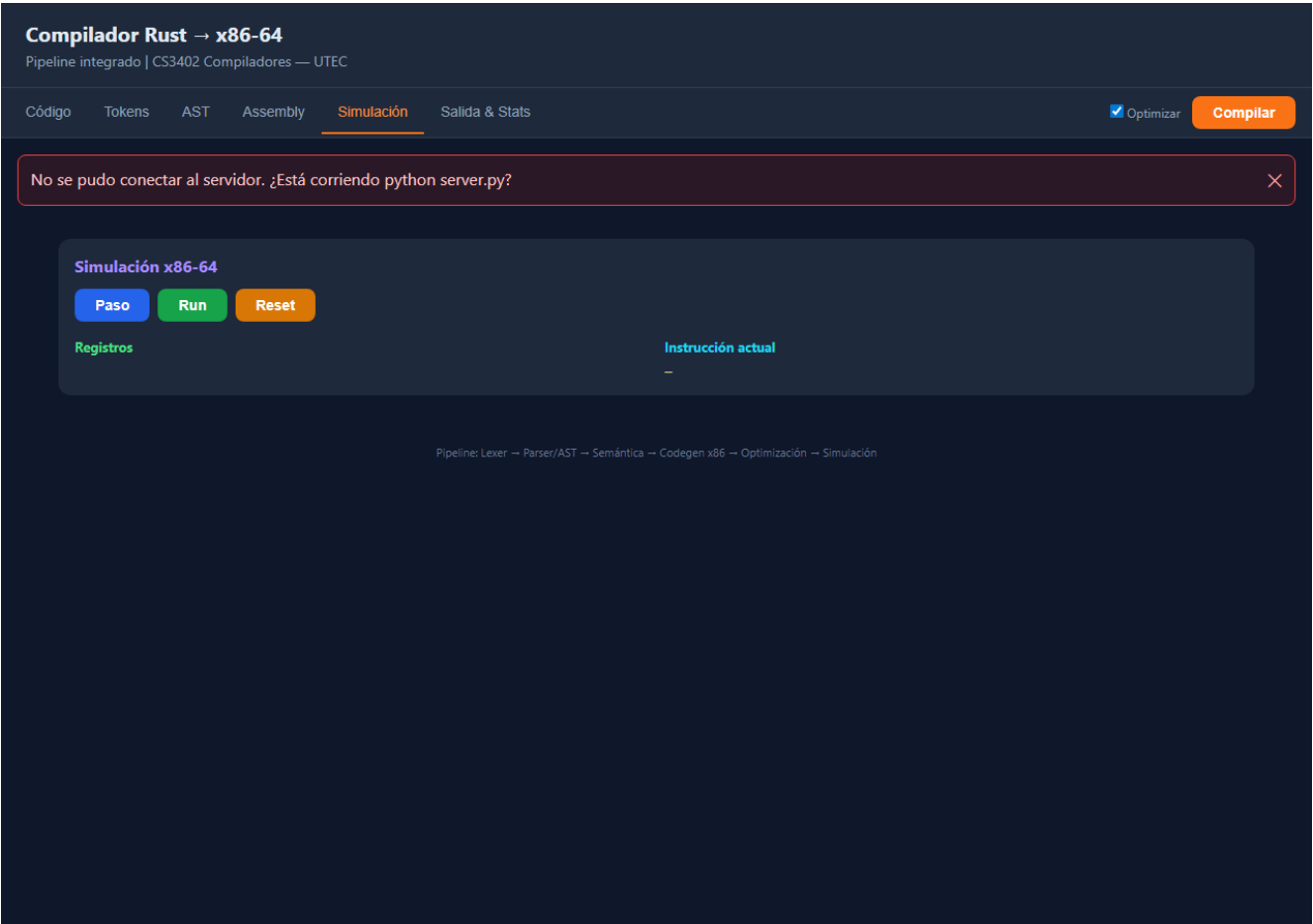
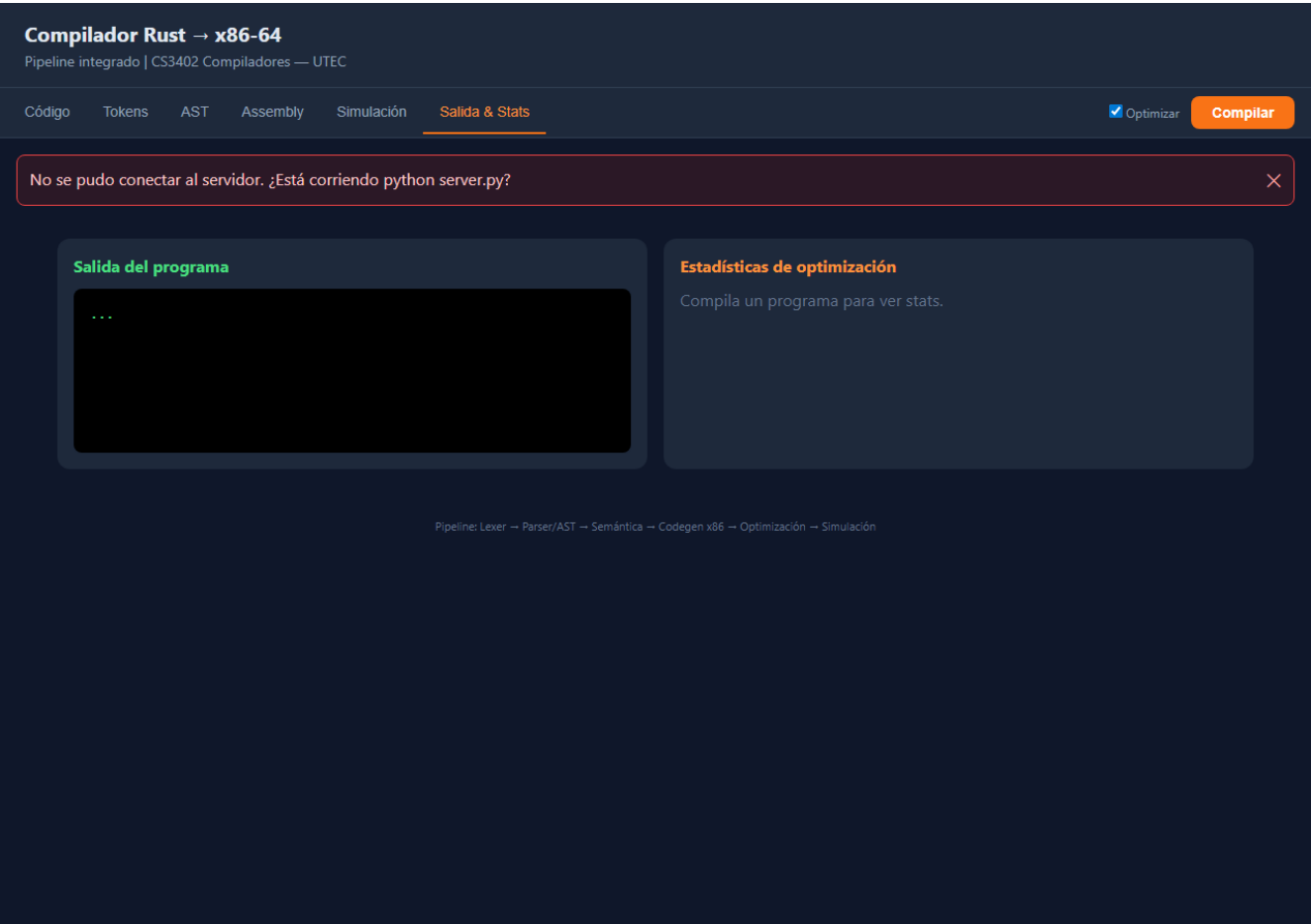


Figura 8. Pestaña Salida y Stats



7. Conclusiones

El compilador cumple las fases requeridas y produce código x86-64 correcto en 21 inputs de prueba. La optimización reduce instrucciones de forma medible. Los benchmarks muestran tiempos de compilación significativamente menores que rustc y GCC en programas equivalentes, coherente con un compilador monolítico de propósito educativo. La aplicación web facilita la demostración en vivo del pipeline completo.

8. Referencias

Aho, Lam, Sethi & Ullman — Compilers: Principles, Techniques, and Tools (2nd ed.). The Rust Programming Language — doc.rust-lang.org. System V AMD64 ABI.